# Параллельные высокопроизводительные вычисления

Краткий отчёт по выполению первого задания:

«Параллельная сортировка Бэтчера»

**Выполнил:**
Куприй Роман Михайлович
523 группа

21 декабря 2023

Москва

# 1. Постановка задачи

Разработать и реализовать алгоритм, обеспечивающий параллельную сортировку методом Бэтчера элементов массива в соответствии с заданным направлением (по возрастанию или по убыванию). Необходимо реализовать сортировку на каждом отдельном процессе и сеть сортировки Бэтчера.

В результате выполнения программы все элементы массива принадлежащие одному процессу отсортированы по возрастанию (убыванию). Каждый элемент массива одного процесса меньше (больше) по сравнению с элементами массива любого процесса с большим рангом, за исключением фиктивных элементов.

# 2. Метод решения

Разработана параллельная программа. Формат команды запуска: *mpirun -np x ./bsort n r*, где $x$ – число процессов, $n$ – количество элементов в упорядочиваемом массиве, а $r$ – 0 либо 1 для сортировки по возрастанию либо убыванию соответственно. На каждом процессе находится одинаковое количество элементов массива. Лишние ячейки при этом заполняются специальным значением `DBL_MAX` либо `-DBL_MAX`.

Для запуска программы использовался скрипт, который отравляется в очередь с помощью планировщика: *sbatch -p test -t 15 -n xp -N xn run.sh*, где $xp$ – число процессов, $xn$ – число аллоцированных узлов.

Для реализации параллельной версии алгоритма, последовательная версия модифицировалась следующим образом: в функцию рекурсивного слияния `S`, добавлено непосредственно слияние двух кусочков массивов, реализованную с участием обменов между процессами и выбором нужных элементов каждым процессом (листинг 3).

Для проверки корректности результата после сортировки вызывается функция `check`, в которой проверяется сортировка каждого локального массива, а затем на процессе с нулевым рангом проверяется глобальная упорядоченность. В случае ошибки в стандартный вывод печатается

предупреждение. Функции приведены в приложении 4.

# 3. Характеристики вычислительной системы

Тестирование выполнялось в очереди *test* системы Lomonosov2, всвязи с высокой загруженностью системы Polus.

- Узлы: 32 узла K40
- CPU: 1x Intel Xeon E5-2697 v3 2.60GHz
- CPU-ядер: 14
- ОЗУ: 64 GB
- Сеть: Infiniband FDR

# 4. Результаты вычислений

Тестирование программы проводилось для трёх массивов размером $1e+6$, $1e+8$ и $1e+9$ элементов. Программа запусукалась по 5 раз для каждой конфигурации, бралось наименьшее время работы. При этом, для массива большего размера, в среднем разброс времени работы программы составляет около 1-2%. Для массива меньшего размера времена работы программы очень маленькие, соответственно разброс больше, поэтому выбиралось минимальное время работы. Подробные результаты показаны в таблицах 1, 2 и 3, а также на графиках 1, 2.

В этих таблицах p/nd – число используемых ядер (mpi процессов) / узлов кластера, T – время сортировки, S и E – ускорение и эффективность, полученные при расчёте, Smax и Emax – максимально возможные эффективность и ускорение, обусловленные степенью внутреннего параллелизма используемого алгоритма. Также в последнем столбце приведены значения числа тактов соритровки Бетчера.

| p/nd | T, sec | S | E | Smax | Emax | Q(p) |
|---|---|---|---|---|---|---|
| 1/1 | 0.0603 | 1.0 | 100% | 1.0 | 100% | 0 |
| 2/1 | 0.0318 | 1.90 | 95% | 2.0 | 100% | 1 |
| 4/1 | 0.0223 | 2.70 | 68% | 3.8 | 95% | 3 |
| 8/1 | 0.0133 | 4.54 | 57% | 6.95 | 87% | 6 |
| 16/2 | 0.0077 | 7.88 | 49% | 12.3 | 77% | 10 |
| 32/3 | 0.0052 | 11.57 | 36% | 21.3 | 66% | 15 |
| 64/5 | 0.0042 | 14.33 | 22% | 36.5 | 57% | 21 |

**Таблица 1.** Результаты замеров времени для входного массива размером 1e+6.

Для расчёта ожидаемого времени, ускорения и эффективности параллельной сортировки Бэтчера:

$$T(n,p) = K\frac{n}{p}(\log_2\frac{n}{p} + \frac{\lceil\log_2 p\rceil\lceil\log_2 p+1\rceil}{2}(1+\frac{tau_s}{K})),$$
$$E(n,p) = (1-\log_n p + \frac{\lceil\log_2 p\rceil\lceil\log_2 p+1\rceil}{2\log_2 n}(1+\frac{tau_s}{K}))^{-1}$$

Здесь, значение $K$ также может быть получено:

$$K = \frac{T_1}{nlog(n)}$$

и составляет $4.37e{-}09$, $2.93e{-}09$ и $2.76e{-}09$ для используемых массивов размеров $10^6$, $10^8$ и $10^9$ соответственно.

Для расчёта максимальной эффективности использовалась формула:

$$Emax = (1+\frac{\log_n p}{2}\log_2\frac{p}{2})^{-1},$$

где $p$ – число процессов, а $n$ – размер входного массива.

| p/nd | T, sec | S | E | Smax | Emax | Q(p) |
|---|---|---|---|---|---|---|
| 1/1 | 5.40 | 1.0 | 100% | 1.0 | 100% | 0 |
| 2/1 | 3.07 | 1.76 | 88% | 2.0 | 100% | 1 |
| 4/1 | 1.98 | 2.73 | 68% | 3.85 | 96% | 3 |
| 8/1 | 1.29 | 4.17 | 52% | 7.18 | 89% | 6 |
| 16/2 | 0.69 | 7.72 | 48% | 13.05 | 81% | 10 |
| 32/3 | 0.45 | 11.9 | 37% | 23.25 | 72% | 15 |
| 64/5 | 0.27 | 19.49 | 30% | 40.9 | 64% | 21 |

**Таблица 2.** Результаты замеров времени для входного массива размером 1e+8.

Полученные результаты показывают хорошую масштабируемость кода – использование 64 процессов не исчерпало потенциал к ускорению.

# 5. Приложение 1

В настоящем приложении приведены реализации используемых функций.

| p/nd | T, sec | S | E | Smax | Emax | Q(p) |
|---|---|---|---|---|---|---|
| 1/1 | 57.3 | 1.0 | 100% | 1.0 | 100% | 0 |
| 2/1 | 33.1 | 1.73 | 86% | 2.0 | 100% | 1 |
| 4/1 | 19.7 | 2.91 | 73% | 3.87 | 97% | 3 |
| 8/1 | 13.4 | 4.25 | 53% | 7.27 | 91% | 6 |
| 16/2 | 7.30 | 7.85 | 49% | 13.33 | 83% | 10 |
| 32/3 | 4.63 | 12.38 | 39% | 23.98 | 75% | 15 |
| 64/5 | 2.79 | 20.48 | 32% | 42.62 | 67% | 21 |

**Таблица 3.** Результаты замеров времени для входного массива размером 1e+9.

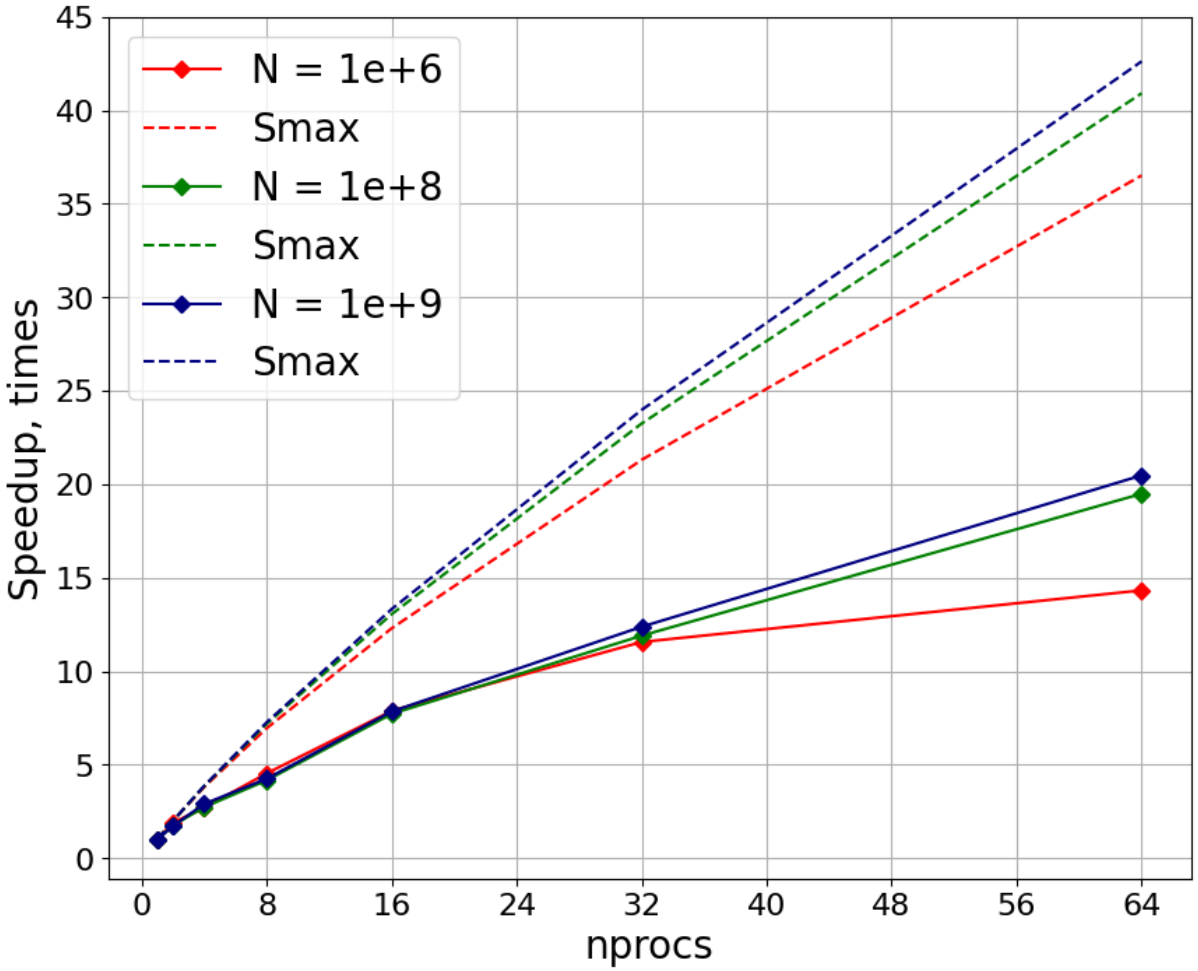



**Рис. 1.** Ускорение

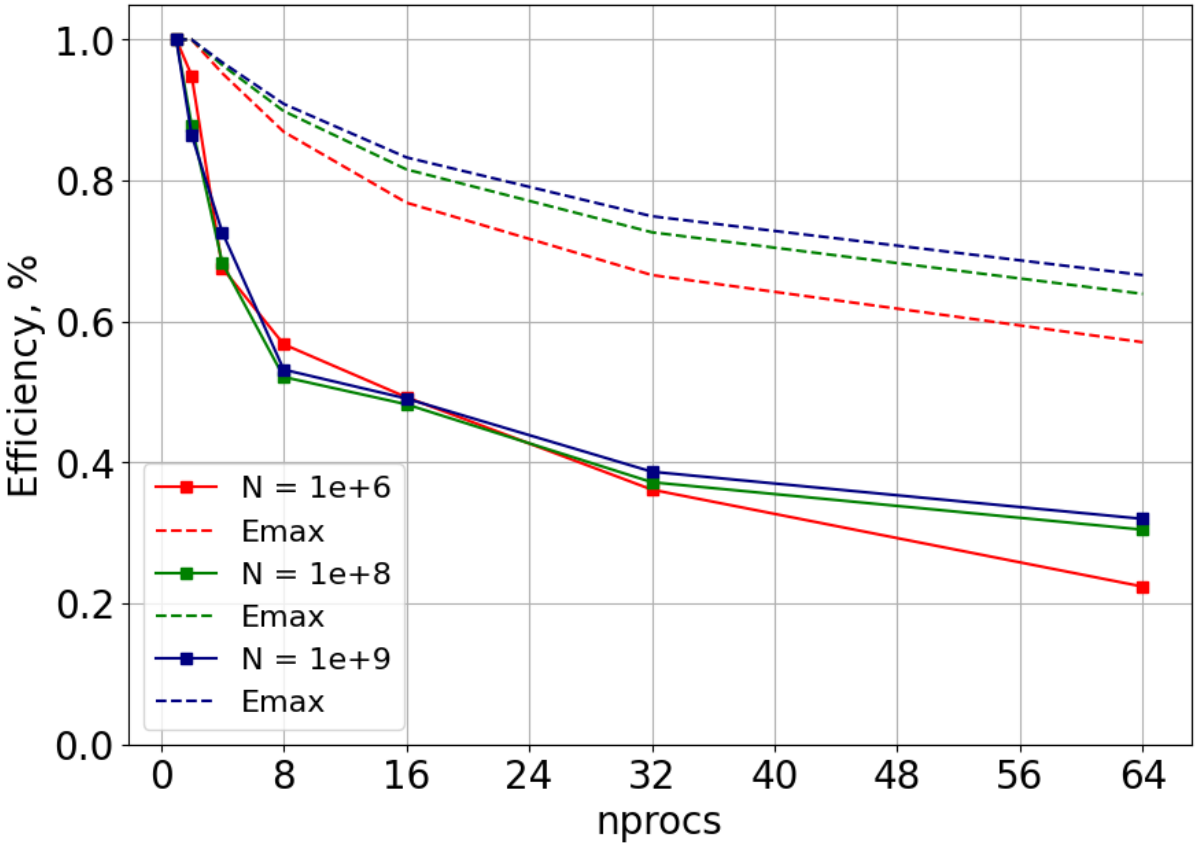



**Рис. 2.** Эффективность ускорения

```cpp
template <typename T>
void SS(std::vector<T>& array, int pos1, int pos2) {
    if (rank != pos1 && rank != pos2) return;

    size_t arr_size = array.size();
    int recv_tacts, pat;
    std::vector<T> recv_array(arr_size);
    std::vector<T> result_array(arr_size);

    if (rank == pos1) par = pos2; else par = pos1;

    MPI_Sendrecv(&tacts_count, 1, MPI_INT, par, 0,
                 &recv_tacts, 1, MPI_INT, par, 0, MPI_COMM_WORLD,
    MPI_STATUS_IGNORE);

    tacts_count = std::max(tacts_count, recv_tacts) + 1;

    MPI_Sendrecv(array.data(), arr_size, MPI_DOUBLE, par, 1,
                 recv_array.data(), arr_size, MPI_DOUBLE, par, 1,
    MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    if (rank == pos2) {
        for (int i = arr_size - 1, j = arr_size - 1, k = arr_size - 1; k
    >= 0;) {
            if (reverse ? array[i] < recv_array[j] : array[i] >
    recv_array[j]) {
                result_array[k--] = array[i--];
            } else {
                result_array[k--] = recv_array[j--];
            }
        }
    } else {
        for (int i = 0, j = 0, k = 0; k < arr_size;) {
            if (reverse ? array[i] > recv_array[j] : array[i] <
    recv_array[j]) {
                result_array[k++] = array[i++];
            } else {
                result_array[k++] = recv_array[j++];
            }
        }
    }

    for (int i = 0; i < arr_size; i++)
        array[i] = result_array[i];

    return;
}
```

**Рис. 3.** Parallel SubMerge

```cpp
bool is_sorted(const std::vector<double>& array, bool reverse) {
    for (size_t i = 1; i < array.size(); i++) {
        if (reverse) {
            if (array[i-1] < array[i])
                return false;
        } else {
            if (array[i-1] > array[i])
                return false;
        }
    }
    return true;
}

bool check(const std::vector<double>& local_array, bool reverse) {
    if (!is_sorted(local_array, reverse))
        return false;

    double first = local_array.front();
    double last = local_array.back();

    std::vector<double> buf(world_size, 0);
    std::vector<double> array(world_size * 2, 0);
    MPI_Gather(&first, 1, MPI_DOUBLE, buf.data(), 1, MPI_DOUBLE, 0,
    MPI_COMM_WORLD);

    for (int i = 0; i < world_size; i++)
        array[i*2] = buf[i];

    MPI_Gather(&last, 1, MPI_DOUBLE, buf.data(), 1, MPI_DOUBLE, 0,
    MPI_COMM_WORLD);

    for (int i = 0; i < world_size; i++)
        array[i*2+1] = buf[i];

    return is_sorted(array, reverse);
}
```

**Рис. 4.** Validation